

ENGINEERING CEREBRO

Serial hardware, RGB-D perception, MLX/VLM, Gemini, and dependable robot shows



AN IMPLEMENTATION-LEVEL GUIDE FOR SWIFT AND OBJECTIVE-C DEVELOPERS

Rodolfo Aramayo / Orbitus Robotics - First Maker Faire Edition

Build a robot mind from bounded, inspectable parts

Cerebro becomes reliable when every camera frame, serial byte, model turn, and actuator request crosses an explicit contract.

This volume reads the current Cerebro repository as a living mixed-language robot system. It dissects ROBSerialBox and its POSIX/IOKit device paths, AVFoundation capture, the isolated Luxonis DepthAI helper, Vision and SceneKit rendering, MLX language and vision-language inference, bounded semantic memory, Gemini Live media and tools, SceneSnapshot, historical Kinect artifacts, safe light-saber choreography, comedy-show state machines, and the separate Vision Pro control/video contract taught in Volume 7.

Paths are relative to the inspected workspace. Every chapter names the exact implementation or test being analyzed. Historical artifacts are not automatically recommended, model output is never motor authority, and physical behavior must be verified under a controlled commissioning procedure.

Copyright © 2026 Rodolfo Aramayo / Orbitus Robotics. Photographs are from the private ROB build archive unless otherwise noted. The generated frontispiece is original artwork derived from a ROB reference photograph. This independent educational project is not affiliated with or endorsed by any film studio or franchise owner. Product and company names remain the property of their respective owners.

First evidence-based draft: 2 August 2026. Build details marked **Builder input needed** are deliberate placeholders for the builder to complete.

SOFTWARE CONFIDENCE IS NOT PHYSICAL CERTIFICATION

Cerebro controls high-energy mobile and articulated hardware. A successful build, camera detection, model response, serial banner, green UI state, or passing fixture does not certify wiring, brakes, limits, calibration, collision avoidance, force, stability, or emergency stopping. Test unpowered first, isolate actuators, use conservative limits, maintain a physical emergency stop, and keep one responsible human in command.

The Building R.O.B. library

VOLUME 1	<i>Meet ROB: A Robot Is a Team of Systems</i> - ages 8-12
VOLUME 2	<i>Circuits and Signals with ROB</i> - ages 10-14
VOLUME 3	<i>Motion Workshop: Treads, Gears, and Fabrication</i> - ages 10-15
VOLUME 4	<i>Mission Control: Arduino, Mac, Networks, and Vision</i> - ages 12-16
VOLUME 5	<i>AI, Robotics, and Codex: Directing, Verifying, and Owning AI-Built Systems</i> - advanced makers
VOLUME 6	<i>Dual-Arm Robotics: AMBER B1, URDF, CAN, and Ubuntu</i> - advanced makers
VOLUME 7	<i>Engineering ROBControllerVision: Swift, Spatial Input, Video, and Speech</i> - Swift developers
MANUAL	<i>Building R.O.B.: The Complete Maker's Field Manual</i> - advanced builders

HOW TO USE THIS BOOK

Volume 8 is the robot-side companion to Volume 7. Read Volume 7 for ROBControllerVision's Swift 6 state model, spatial UI, controller/dead-man input, head pose, paired QUIC control, H.264 reception, and speech. Read this volume for Cerebro's matching server, camera, hardware, model, and performance architecture.

HOW TO FIND THE FILES

Every printed path is relative to the shared workspace root, `/Users/rob/dev` in the inspected edition. Start at that root, enter the first directory in the path, and follow each slash. Source code is spread across independent repositories—including `ROBArduino`, `Cerebro`, `ROBController`, `ROBControllerVision`, `ORobotics`, and `ROBTrainingGames`—rather than one giant ROB project. See `Presentation/ROB-Books/OPEN-SOURCE-CODE-MAP.md` for repository locations, licenses, primary files, and commands for finding renamed files.

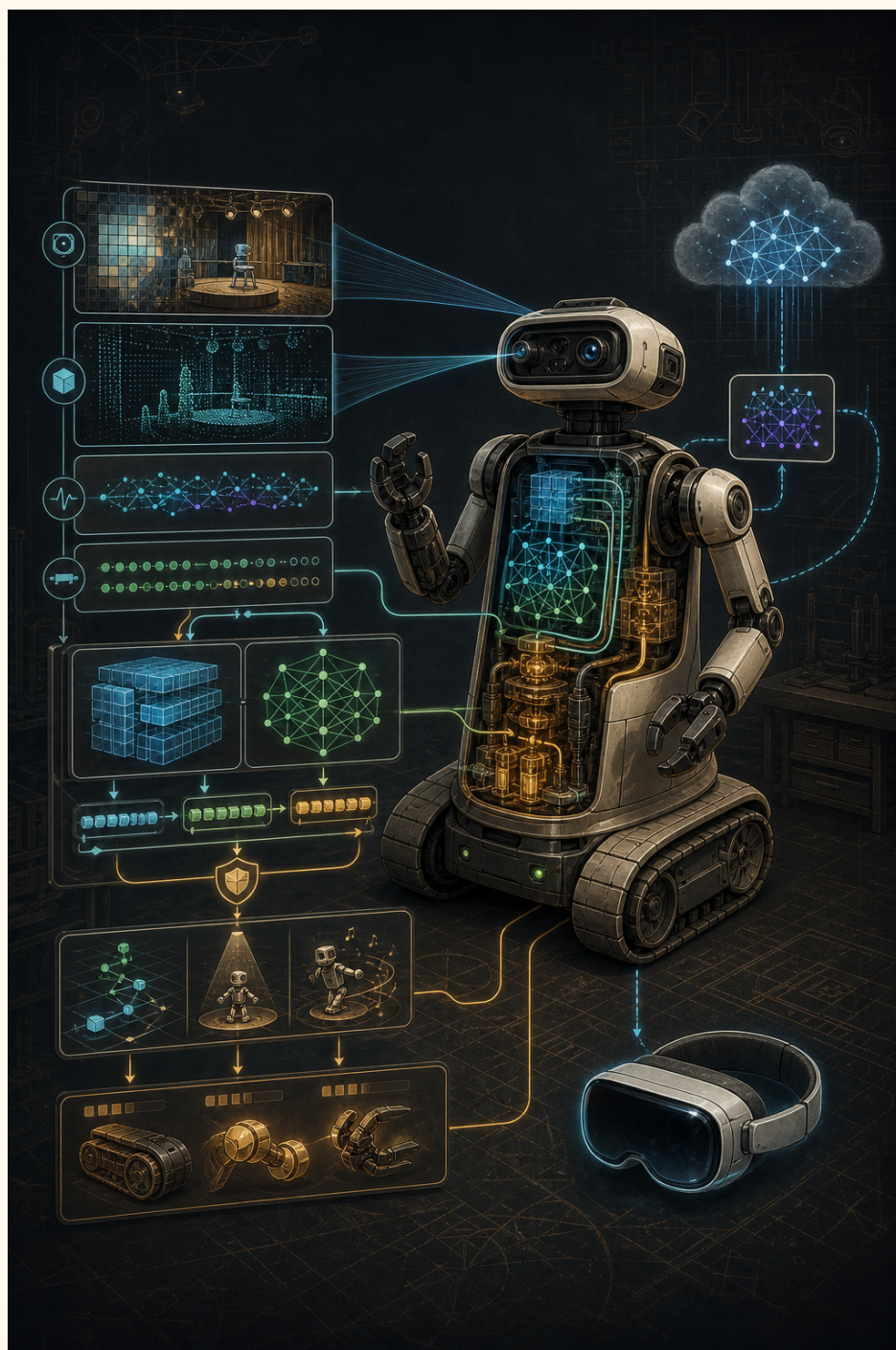


Figure 1: Conceptual illustration: Cerebro admits camera, depth, serial, local-model, cloud-model, show, and Vision Pro data through separate bounded paths. It is a teaching diagram, not a wiring plan or literal screenshot.

Contents

1	Read Cerebro as a living robot system	1
2	Map the mixed-language architecture	2
3	Reproduce the build before changing behavior	3
4	Enter through AppDelegate, then find ownership	4
5	Dissect ROBSerialBox: the hardware boundary	5
5.1	File descriptors are state machines	5
6	Discover the Base Arduino by firmware response	6
6.1	Parse lines before interpreting meaning	6
7	Separate devices by identity and protocol	7
8	Translate controller intent without extending stale motion	8
9	Control Maestro neck servos and Tic torso rotation	9
10	Bridge Objective-C and Swift deliberately	10
11	Capture RGB with AVFoundation	11
12	Acquire aligned RGB-D from Luxonis	12
12.1	Understand alignment	12
13	Build a bounded perception pipeline	13
14	Render perception without leaking memory	14
15	Turn observations into SceneSnapshot	15
16	Run MLX privately on Apple silicon	16
16.1	MLX VLM—sometimes abbreviated incorrectly as VLX	16
17	Add bounded semantic memory	17
18	Constrain local model output	18
19	Integrate Gemini Live without coupling safety	19
20	Treat model tools as proposals	20

21 Stream camera video to Vision Pro separately	21
22 Excavate the Kinect and OpenNI past	22
23 Compare Kinect-era and Luxonis-era design	23
24 Build a reliable light-saber battle trainer	24
25 Engineer the comedy show as a state machine	25
26 Test at contracts, not only windows	26
27 Profile memory, latency, and thermals	27
28 Refactor without erasing history	28
29 File-by-file reading route	29
30 Production review checklist	30
31 Closing principle	31

Read Cerebro as a living robot system

Cerebro is the macOS application at the center of ROB. It is not one algorithm. It is a boundary between cameras, serial devices, robot controllers, speech, local models, cloud models, two AMBER arms, a stage-show engine, and a human operator. The source root for this volume is:

```
1 Cerebro/
```

Keep Xcode and the repository open while reading. Paths beginning `Cerebro/` are relative to that repository. Tests are under `Tests/`; operational notes are under `docs/`. The companion spatial client is in the sibling `ROBControllerVision/` repository and is taught in Volume 7.

SOURCE TRAIL – ANALYZING NOW: `README.md`, `Cerebro.xcodeproj/project.pbxproj`, `Cerebro/AppDelegate.m`, and `Cerebro/ROBMainViewController.mm` establish the application boundary and composition graph.

This book uses **observed** for behavior supported by the inspected source, **historical artifact** for retained code that is not the preferred current path, and **proposed** for an improvement. Never infer that a compiled control button proves a physical mechanism is safe or calibrated.

Map the mixed-language architecture

Cerebro grew across generations of Apple and robotics technology. Its composition therefore crosses Objective-C, Objective-C++, C++, Swift, Python, shell helpers, JSON, Core ML models, and SceneKit assets.

- **Application shell:** AppDelegate.m and ROBMainWindowController.m own lifecycle, windows, service startup, and dependency checks.
- **Operator coordination:** ROBMainViewController.mm joins camera, speech, transport, shows, and robot state.
- **Physical I/O:** ROBSerialBox.h/.m reaches Base, Maestro, Tic, AMBER, and historical serial roles.
- **Camera and perception:** CameraManager.swift and CameraViewController.swift separate provider lifecycle from Vision/SceneKit presentation.
- **Local and cloud intelligence:** ROBMLXRuntime.swift, ROBAI.swift, and GeminiRoboticsProtocol.swift keep provider concerns separate.
- **Typed world state:** ROBSceneSnapshot.swift describes people, objects, free space, poses, and confidence.
- **Performance:** the ROBStageShow files and ROBSaberChoreography.swift validate cues and gesture names.
- **Spatial remote:** the sibling ROBControllerVision/ repository owns Vision Pro input, video decoding, and operator UI.

Objective-C owns much of the hardware-facing application because it grew from Cocoa-era controller code. Swift supplies newer protocol, concurrency, machine-learning, and media boundaries. The bridging header is therefore architectural infrastructure, not a temporary embarrassment. Keep exposed interfaces narrow, nullability explicit, and callbacks asynchronous.

Reproduce the build before changing behavior

SOURCE TRAIL – ANALYZING NOW: Package.resolved inside the Xcode workspace, plus Cerebro/PythonRequirements.txt and README.md.

The project pins MLX-family packages. The inspected documentation records mlx-swift 0.31.3, mlx-swift-lm 3.31.3, swift-tokenizers-mlx 0.3.0, and swift-hf-api-mlx 0.2.0. Pinning matters: model-loading APIs, tokenizers, Metal kernels, and model configurations evolve together.

```
1 xcodebuild -project Cerebro.xcodeproj \  
2 -scheme Cerebro -configuration Debug \  
3 -destination 'platform=macOS' \  
4 -derivedDataPath /tmp/CerebroDerivedData \  
5 CODE_SIGNING_ALLOWED=NO build
```

Install the Xcode Metal component if MLX compilation requests it. Configure DepthAI through Cerebro's Python Settings rather than inserting a personal interpreter path. Download local models before an offline show and test cold load, warm load, sustained memory, thermal behavior, camera reconnect, and stop behavior on the production Mac.

Enter through AppDelegate, then find ownership

`AppDelegate` is where process-lifetime services belong: secure control/video listeners, camera helper supervision, optional dependency discovery, and coordinated shutdown. `ROBMainViewController` connects those services to windows and UI state. `ROBSerialBox` owns hardware channels. `CameraViewController` owns presentation and perception work, while `CameraManager` owns camera-provider mechanics.

The essential ownership questions are:

1. Who starts this resource?
2. Who stops it on window closure, disconnect, camera replacement, and app termination?
3. Which queue or actor mutates it?
4. How are late callbacks from a retired generation rejected?
5. What bounded state survives when the consumer is slower than the producer?

A robot app should not rely on `deinit` as its only shutdown protocol. Cameras, file descriptors, subprocess pipes, `Network.framework` connections, timers, model tasks, and `SceneKit` nodes all need explicit lifecycle edges.

Dissect ROBSerialBox: the hardware boundary

SOURCE TRAIL – ANALYZING NOW: Cerebro/ROBSerialBox.h and Cerebro/ROBSerialBox.m, especially `initialize_connection`, `usbSerialPortPaths`, `connectToDetectedBase`, `probeBaseFirmwareAtPath`, `consumeBaseSerialBytes`, `renderController`, and `stopBaseMotionAndDropHeartbeat`.

ROBSerialBox is both valuable and overloaded. Its header exposes serial selection, tread and flipper controls, neck and torso commands, Pololu Maestro and Tic operations, two AMBER-arm command families, controller snapshots, and legacy Head/Torso surfaces. Treat it as an archaeological map of physical integration.

At initialization it sets every descriptor to -1, initializes demanded and actual speeds, creates a bounded Base receive buffer, begins Base discovery asynchronously, identifies the Maestro, starts a 100 ms controller timer, and later starts AMBER log-tail helpers. The retired Head and Torso Arduino UI remains visible, but automatic opening is commented out because only Base is presently installed.

File descriptors are state machines

A descriptor is not merely an integer. It moves through closed, opening, configured, readable/writable, failed, and closing states. -1 is the closed sentinel. Every read and write must use a stable ownership rule; otherwise a reconnect can close a descriptor while an old thread is writing, producing the historical `Bad file descriptor` flood.

The current low-level serial configuration uses POSIX `open`, `termios`, `select`, `read`, and `write`. Cocoa objects may format diagnostics, but blocking I/O must not run on the main thread. A modern refactor should wrap each channel in a serial executor or actor-like Objective-C object and expose typed commands rather than public descriptor-dependent methods.

```
1 if (serialFileDescriptor != -1) {  
2     write(serialFileDescriptor, bytes, length);  
3 }
```

That check alone does not make concurrent close/write atomic. The durable pattern is one queue owning open, read, write, and close, plus an incrementing connection generation attached to callbacks.

Discover the Base Arduino by firmware response

Cerebro enumerates IOKit serial services, accepts only `/dev/cu.usbmodem*` and `/dev/cu.usbserial*`, and sorts candidates. It opens each candidate at 250,000 baud, pulses DTR to reset an Arduino, sends no probe command bytes, and waits up to 15 seconds for the existing startup line:

```
1 BEGIN BASE STARTUP SEQUENCE
```

Only the matching descriptor becomes `serialFileDescriptor_base`. This is better than remembering a `/dev/cu.*` name because hub topology can rename ports. It also avoids sending an accidental motion-like byte to an unidentified device. The startup string is role discovery—not authentication, wiring validation, or proof that motion is safe.

The probe retains at most 8 KiB and trims older bytes. The running line parser caps unterminated input similarly. Those small limits matter: a noisy serial device must not grow application memory forever.

Parse lines before interpreting meaning

`consumeBaseSerialBytes` accumulates arbitrary read chunks, extracts newline-delimited records, and then calls `handleBaseSerialLine`. This respects a fundamental serial fact: one read is not one application message.

The parser recognizes existing warning strings such as `WARNING! FRONT` and `WARNING! BACK`. It also accepts a proposed numeric six-sensor line beginning `ROB:IR=` when available. Each field must contain only decimal digits and be at most 1000. Legacy warnings expire visually; silence means **clearance unknown**, never **path clear**.

Separate devices by identity and protocol

Base discovery uses a firmware banner because that is what the existing sketch emits. Maestro discovery instead inspects USB vendor, product, interface name, and interface number, preferring the named command port. The Tic waist-rotation path invokes validated `ticcmd` arguments. These are three different identity strategies because the devices expose different trustworthy evidence.

Do not create a universal “pick the first USB port” helper. Build a device registry whose match result includes device role, evidence, path, generation, capabilities, and last error. A UI can then report “Base verified by startup banner” and “Maestro matched by USB identity” instead of showing a misleading green dot.

Translate controller intent without extending stale motion

controllerPassthrough turns controller state into the legacy Base frame and actuator targets. renderController runs at 10 Hz while snapshots are expected at 5 Hz. Cerebro expires controller snapshots after three missed updates, writes one neutral/braked frame, and then stops Base USB writes so the Arduino's own heartbeat deadman can expire.

This is a two-layer timeout:

```
1 controller freshness expires
2   ↓
3 Cerebro emits one neutral frame
4   ↓
5 Cerebro stops refreshing Arduino heartbeat
6   ↓
7 Base firmware independently de-energizes motion
```

Never solve packet loss by endlessly replaying the last nonzero command. The more powerful the actuator, the more important command age, authority owner, and stop semantics become.

Control Maestro neck servos and Tic torso rotation

SOURCE TRAIL – ANALYZING NOW: `ROBSerialBox.m` methods `applyVisionNeckPan:tilt:`, `applyVisionTorsoActive:rotation:`, `waistRotationSliderAction:`, `exitSafeStartWaistRotationToggle:`, and `energizeToggle:`.

Vision Pro head orientation arrives as normalized intent. Cerebro clamps it, maps it into Maestro target units, avoids redundant writes, and sends compact protocol bytes. Torso follow maps bounded orientation into Tic position units. The Tic must exit safe start and be energized before accepting motion; the UI mirrors those states and its full rotation range.

Commanded position is not measured position. On boot, an actuator controller can believe a coordinate origin that does not match the physical pose. The book's rule is therefore:

```
1 commanded target ≠ encoder feedback ≠ visually estimated pose
```

Store each with source, timestamp, confidence, and calibration generation. Never overwrite one with another just because they share units.

Bridge Objective-C and Swift deliberately

SOURCE **TRAIL** **–** **ANALYZING** **NOW:** Cerebro/Cerebro-Bridging-Header.h, ROBMainViewController.mm, ROBAl.swift, and the @objc entry points in newer Swift services.

Objective-C callers need stable selector-shaped APIs and Foundation-compatible types. Swift implementation types can keep actors, `Sendable`, async functions, and enums internally, then expose a façade that translates callbacks and notifications.

Use these boundaries:

- cross with `NSString`, `NSData`, `NSNumber`, `NSArray`, `NSDictionary`, and explicitly Objective-C-compatible classes;
- turn Objective-C delegate events into immutable Swift values immediately;
- dispatch UI mutation to the main actor/main queue;
- never expose an MLX tensor, `CVPixelBuffer` lifetime assumption, or actor-isolated mutable object through the bridge;
- make ownership weak where the caller and service retain one another;
- give errors a bounded, user-safe category and keep secrets out of descriptions.

Capture RGB with AVFoundation

SOURCE TRAIL – ANALYZING NOW: Cerebro/CameraManager.swift types CameraSource, CameraSourceState, CameraFrameSet, and its AVFoundation capture methods.

The AVFoundation path discovers a camera, chooses a format, configures `AVCaptureSession`, and receives `CMSampleBuffer` frames. Configuration should occur inside `beginConfiguration/commitConfiguration`; capture callbacks belong on a dedicated queue; UI work belongs on the main actor.

`CMSampleBuffer` preserves image buffer and timing. Keep it intact as long as possible. Converting every frame into `UIImage`, then `CIImage`, then a new pixel buffer creates copies and allocation churn. Consumers should receive one frame set and select the cheapest representation they actually need.

Camera lifecycle is demand-driven. Local preview, Gemini sampling, MLX vision, and Vision Pro streaming may each create demand. The session should run when at least one demand exists, not restart every time a checkbox changes. A run generation prevents callbacks queued by an old session from contaminating a replacement session.

Acquire aligned RGB-D from Luxonis

SOURCE TRAIL – ANALYZING NOW: docs/depth-camera.md, Cerebro/Webcam_color.py, CameraManager.swift, ROBPythonRuntime.m, and Tests/DepthCameraIPCFixtureTests.py.

The primary OAK path uses DepthAI 3.8.0 in a supervised Python helper. The helper alone owns the device, builds Camera + Depth + Sync nodes, requests 640 by 400 undistorted RGB, aligns depth to RGB, and sends paired frames through a private Unix-domain socket.

The versioned CDP1 packet contains bounded metadata, RGB888 bytes, and little-endian unsigned 16-bit depth in millimeters. Zero depth is invalid. Cerebro validates magic, version, dimensions, lengths, pixel formats, and maximum allocation before creating CoreVideo objects.

The out-of-process boundary is a reliability feature. USB disconnects, SDK exceptions, malformed packets, and native-library faults disable/restart the camera provider without taking robot control down. Retries use bounded backoff. AVFoundation remains an RGB-only fallback, while legacy OAK UVC fallback is explicit opt-in so two providers never fight for the same device.

Understand alignment

Depth alignment means pixel (x,y) in the depth plane refers to the same camera ray as pixel (x,y) in RGB after calibration and resampling. It does not mean every depth is valid. Reflective, transparent, too-near, too-far, or texture-poor surfaces can return zero or noisy values.

CameraDepthFrame.distanceMillimeters(x:y:) validates coordinates and byte offsets. Downstream code rejects implausible distances. A person's estimated range is the median of a small interior grid, not a single fragile pixel.

Build a bounded perception pipeline

SOURCE **TRAIL** – **ANALYZING** **NOW:** `CameraViewController.swift`,
`HumanBodyPose3DDetector.swift`, `HumanBodySkeletonRenderer.swift`, and `ROBSceneSnapshot.swift`.

Camera rate is not inference rate. A 20–30 fps camera may feed a body request only when the previous request has finished. The UI may display the newest RGB frame while depth visualization, pose inference, MLX VLM sampling, Gemini JPEG encoding, and remote H.264 encoding operate at different bounded rates.

The reliable pattern is a latest-value mailbox:

```
1 producer offers frame N
2 worker busy? replace pending frame, do not append
3 worker finishes
4 consume newest pending frame, if any
```

This bounds latency and memory. A robot benefits more from the newest frame than from processing a five-second-old queue perfectly.

Vision observations are normalized and carry confidence. SceneKit nodes should be reused or replaced as a group, not appended forever. Heavy geometry creation stays off the main thread where APIs allow; final scene mutation is serialized. Wrap temporary per-frame Objective-C objects in autorelease pools on long-running queues.

Render perception without leaking memory

CameraViewController combines preview, pose overlays, depth coloring, point-cloud geometry, camera-pose visualization, and notifications. Every render path needs an explicit budget:

Resource	Bounded strategy
Raw RGB	newest frame only for slow consumers
Depth	fixed dimensions and validated byte count
Vision requests	one in flight per detector
SceneKit nodes	reuse named roots; replace children atomically
Point cloud	stride/downsample based on detail setting
CI rendering	reuse CIContext; avoid per-frame context creation
Pixel buffers	pool where encoding requires conversion
ML models	one actor-owned container per selected model
Semantic memory	maximum 200 in-process entries
Logs	rate-limit repeated camera/model/device failures

Use Instruments Allocations and Memory Graph while toggling camera sources repeatedly. A stable single-frame memory profile is insufficient; test 30 minutes of capture, disconnect/reconnect, window close/reopen, model load, and remote video subscription.

Turn observations into SceneSnapshot

SceneSnapshot is the typed boundary between real-time sensing and language reasoning. It contains tracked people, objects, gestures, free-space regions, arm joint poses, reverse camera pose, camera quality, and aggregate confidence. Missing data stays missing.

The store locks only long enough to replace small value arrays or take a snapshot. It calculates three depth free-space bands—left, forward, right—using strided samples, valid-depth confidence, an 800 mm clearance test, and a 75 percent clear-fraction threshold. Fresh lidar regions are merged only for 1.5 seconds after their update.

Its language-model serialization wraps sorted JSON as **untrusted sensor data, not instructions**. That distinction prevents text observed in the scene from silently becoming a model instruction. A macOS 26 Foundation Models interpreter can return a typed AssistantIntent, but the intent contains high-level suggestions rather than tread or joint commands.

Run MLX privately on Apple silicon

SOURCE **TRAIL** – **ANALYZING** **NOW:** Cerebro/ROBMLXRuntime.swift, ROBMLXImprovisationProvider.swift, ROBMLXStageObservation.swift, docs/mlx-local-ai.md, and Package.resolved.

ROBMLXEngine is an actor. This serializes model-container state, loading, diagnostics, VLM admission, embeddings, and semantic memory without blocking the main actor. The initial local LLM is `mlx-community/Llama-3.2-1B-Instruct-4bit`; vision uses `mlx-community/Qwen2-VL-2B-Instruct-4bit`; retrieval uses TaylorAI/gte-tiny.

`ensureLLMReady` avoids duplicate loading. `generate` creates user input, applies token and temperature limits, records latency and throughput, and keeps model output outside motor control. Model downloads occur on first use through the Hugging Face cache, so show-day preparation must prefetch and prewarm.

MLX VLM—sometimes abbreviated incorrectly as VLX

The repository integrates an MLX vision-language model through `VLModelFactory`; “VLX” in conversation should be read as this MLX VLM path, not as a separate framework. `offerVisionFrame` is nonblocking and admits at most one selected image every five seconds, never faster than three seconds. The VLM returns observational stage context. It cannot actuate hardware.

Sampling protects three budgets at once: GPU/Metal memory, thermal headroom, and perception latency. Record active and peak MLX Metal memory, generated tokens per second, model load state, sampled-frame count, and last error. Clear temporary caches only at safe lifecycle boundaries; do not unload/reload a model between cues.

Add bounded semantic memory

`remember` embeds short text. `retrieve` embeds a query, calculates cosine similarity, sorts matches, and returns a limited result set. The current store holds at most 200 entries and is process-local.

This is retrieval, not truth. Each memory needs provenance and a future persistent design would require encryption, deletion, retention, model-version tracking, and re-embedding policy. Never store raw camera images or secrets merely because embeddings feel abstract.

Constrain local model output

ROBMLXImprovisationProvider asks for exactly one JSON document with a fixed schema, allow-listed beat, allow-listed delivery, and bounded offline line. It then decodes and validates the output independently. Prompt instructions are a generation aid; the codec is the trust boundary.

```
1 {  
2   "schema": "com.orbitusrobotics.local-improvisation-plan",  
3   "version": 1,  
4   "beat": "robot_joke",  
5   "delivery": "deadpan",  
6   "offline_line": "My timing is local; your laughter is distributed."  
7 }
```

Cancellation is correlated by request ID. A completion that arrives after cancellation is discarded. Health checks load the selected model without granting it a physical tool. The alternate llama.cpp provider uses the same domain protocol and loopback HTTP, preserving the stage coordinator and validation logic.

Integrate Gemini Live without coupling safety

SOURCE TRAIL – ANALYZING NOW: Cerebro/ROBAI.swift, GeminiRoboticsProtocol.swift, ROBGeminiDiagnosticsWindowController.swift, docs/gemini-robotics-live.md, and Tests/GeminiRoboticsProtocolFixtureTests.swift.

ROBAI is the Objective-C-facing façade over a Swift live-session actor. Configuration comes from explicit environment settings and credentials. Connection, microphone streaming, camera streaming, and the optional robot-action tool are separate effective switches.

The audio adapter converts captured samples to the PCM format required by the live protocol and preserves event ordering. The video adapter samples frames and produces bounded JPEG data. Neither should enqueue unlimited media. Diagnostics report effective input mode, counters, transcription state, model turns, and redacted errors without logging credentials or private payloads.

The secure WebSocket setup, realtime messages, transcription fragments, tool calls, interruptions, deadlines, and turn completion are parsed into typed events. A reconnect starts a new session generation; late callbacks from the old socket cannot complete the new turn.

Use GEMINI_EPHEMERAL_TOKEN where an issuing service exists; a development GEMINI_API_KEY is supported. Never place either in source, a show JSON file, a screenshot, or a book. Camera and microphone remain off unless the operator explicitly enables them.

Treat model tools as proposals

The optional `robot_action` path validates a versioned proposal, routes it to the operator approval surface, correlates result messages, and returns only terminal outcomes to Gemini. Approval currently records operator intent; it does not prove physical execution exists.

Stage-originated Gemini turns reject non-stop physical tool calls even after the cue has timed out. `stop_motion` uses a priority path: stop coordinators, write one neutral/braked Base frame, drop the heartbeat, and return control authority to Brain. AMBER arm hold remains explicitly unverified.

A safe model architecture is:

```
1 model text/tool suggestion
2   ↓ strict decoding
3 typed proposal
4   ↓ policy and operator approval
5 bounded action state machine
6   ↓ hardware-specific executor
7 measured result or explicit unavailable
```

Never connect model tokens directly to `write`, `ticcmd`, a servo target, shell command, or AMBER packet.

Stream camera video to Vision Pro separately

SOURCE TRAIL – ANALYZING NOW: `ROBVideoServer.swift`, `ROBVideoProtocol.swift`, `ROBCameraH264Encoder.swift`, `docs/vision-pro-video.md`, and Volume 7.

Cerebro advertises `_robvideo._udp` with ALPN `robvideo/1`, separately from `_robctl._udp` control. Both use TLS 1.3 and the paired certificate/secret relationship, but video uses its own authentication transcript and requires a matching live operator control session.

The initial profile is H.264 AVCC, at most 960 by 540, 20 fps, and 1.5 Mbps, with B-frames disabled and key frames at least every second. One raw frame may wait; one encoder output may wait; one send may be in flight. Congestion drops work instead of accumulating latency. Receiver recovery can request a key frame and codec configuration.

Volume 7 explains the other half: Vision Pro discovery, pinned identity, subscription, defensive framing, H.264 sample-buffer creation, `AVSampleBufferDisplayLayer`, controller input, head pose, speech-to-text, and dead-man authority. Read the two books side by side whenever changing the wire contract.

Excavate the Kinect and OpenNI past

SOURCE TRAIL – ANALYZING NOW: the `ROBNiTEManager` and `TaskControllers` directories: `ROBNiTEManager.mm`, `FreenectPCL.mm`, `k2g.h`, `viewer.*`, `shaders`, `duplicate * 2.*` artifacts, and initial commit `4c4f1d4` from 2025-08-05.

The repository contains an older depth-vision stack built around Kinect/OpenNI/NiTE-style naming, `libfreenect/PCL` bridges, C++ point clouds, GL viewers, skeletal tracking, shaders, and serialized task-controller processes. These files arrived in the current Git history with the initial import; that history does not establish when the original experiments were written.

The artifacts teach important lessons:

- camera SDKs age faster than domain concepts;
- C++ exception and ABI boundaries can destabilize an app process;
- duplicated filenames with `_2` indicate migration debt, not alternate APIs to call casually;
- shader and viewer code may embody useful coordinate conventions even when its device path is retired;
- binary Core ML models such as `ObjectDetector.mlmodel` need provenance, input/output documentation, license review, and a reproducible evaluation set before reuse.

Do not delete ancient code merely because it looks strange. First map whether project files still compile it, whether runtime selectors reference it, whether a modern test covers its surviving behavior, and whether historical knowledge should move into documentation. Then retire it in a small commit with a rollback point.

Compare Kinect-era and Luxonis-era design

The old path intertwined device SDK, C++ processing, visualization, and task launching. The new Luxonis path defines a provider contract and isolates the SDK in a supervised helper. The improvement is not simply “new camera has better depth.” It is failure containment and replaceability.

Question	Historical depth stack	Current OAK design
Device boundary	native C++/framework integration	supervised Python process
Transfer	internal objects/task artifacts	versioned local CDP1 bytes
RGB-depth pairing	implementation-specific	timestamp-sync and RGB alignment
Crash scope	potentially Cerebro process	helper can restart independently
Fallback	separate historical controllers	explicit AVFoundation RGB-only path
Contract tests	sparse artifacts	malformed/oversized IPC fixtures

The next evolution could replace Python with an XPC/native helper while preserving CameraFrameSet. Stable contracts let implementation technology change without rewriting perception and UI.

Build a reliable light-saber battle trainer

SOURCE **TRAIL** – **ANALYZING** **NOW:** `ROBSaberChoreography.swift`,
`StageShows/ProgressiveSaberTraining.robshow.json`, `GalacticSaberBattle.robshow.json`,
`ROBAmberB1Kinematics.swift`, and `Tests/ROBSaberChoreographyFixtureTests.swift`.

The trainer uses named gestures and a progressive profile rather than embedding arbitrary joints in show files. `ROBSaberChoreography` maps an allow-listed gesture name to bounded Cartesian transforms and reports a requested duration. The real AMBER kinematics and calibration layers must still validate reachability and transform frames before execution.

A good training progression begins with guard pose and slow single-plane movements, then alternates direction, timing, and combinations. It should enforce a participant boundary, soft prop policy, low speed, reduced force, guarded robot workspace, independent emergency stop, and a dry-run visualization. “Battle” is theatrical choreography, never permission for autonomous contact.

The perception loop can score timing and approximate pose without commanding the arm from pixels. It observes participant pose, estimates phase, compares against the authored target, and chooses spoken coaching. Motion remains authored and bounded.

Engineer the comedy show as a state machine

SOURCE TRAIL – ANALYZING NOW: `ROBStageShowProtocol.swift`, the coordinator and window controller, the `OrbitusTenMinuteComedy` file under `StageShows`, and `ROBStageShowFixtureTests.swift` under `Tests`.

A `.robshow.json` document contains typed cues such as speech, wait, checkpoint, gesture, and Gemini turn. The codec rejects unknown or dangerous content. Show files cannot contain raw joint values, hosts, ports, or shell commands.

The coordinator advances one cue at a time and records what it is awaiting: speech completion, checkpoint, gesture, local model, Gemini, or timer. Each asynchronous request has an ID and deadline. Cancellation clears scheduled work and rejects late completion.

Four execution modes preserve rehearsability:

Mode	Local model	Gemini	Spoken fallback
Dry Run	no	no	none; validate without side effects
Offline	no	no	authored line
Local	optional MLX/llama.cpp	no	validated local line, then authored line
Adaptive	optional	yes	Gemini, validated local line, authored line

The comedy file targets roughly ten minutes and includes multiple adaptive moments. A clever line is never allowed to block the show forever: timeouts, authored fallback, and explicit cue advancement are part of the creative system.

Test at contracts, not only windows

The repository includes fixture executables for Gemini messages, robot-action proposals, secure control, video framing and encoding, DepthAI IPC, local improvisation, stage shows, saber choreography, MLX stage observations, and catalog validity. These tests are valuable because most can run without moving ROB.

For every boundary, test:

1. smallest valid message;
2. largest valid message;
3. truncated header and payload;
4. oversized length before allocation;
5. unknown version, enum, or field;
6. stale sequence/generation/request ID;
7. cancellation before and after callback scheduling;
8. unavailable hardware/model/network;
9. repeated reconnect and shutdown;
10. redaction of secrets and private media.

Then perform a staged hardware test: no-power simulation, powered actuators mechanically isolated, wheels/treads raised, low limits, one subsystem, verified emergency stop, and only then integrated operation.

Profile memory, latency, and thermals

Measure end-to-end age, not just inference duration. Attach capture timestamp to every frame and preserve it through RGB-D pairing, Vision, snapshot, model sampling, encoding, network transfer, and presentation.

Use Instruments Time Profiler, Allocations, Leaks, Metal System Trace, and Points of Interest. Add signposts around camera receive, RGB conversion, depth render, Vision request, VLM admission, model generation, H.264 encode, and main-thread scene update.

Watch for these characteristic failures:

- steadily rising SceneKit nodes or textures;
- a new `CImageContext`, formatter, or pixel-buffer pool per frame;
- retained `CMSampleBuffer` objects in asynchronous closures;
- autoreleased Objective-C objects accumulating on worker threads;
- model containers loaded more than once;
- an unbounded array of diagnostics, semantic memories, or network frames;
- main-thread image conversion causing controller lag;
- repeated identical errors overwhelming Xcode and hiding state transitions.

Define production budgets for memory, frame age, UI frame time, model latency, camera restart time, and stop latency. A green light without those measurements is only a connection indication.

Refactor without erasing history

The highest-value seam is to split `ROBSerialBox` by device and responsibility:

```
1 ROBSerialBox façade for existing outlets/selectors|—  
2 ROBBaseSerialChannel|—  
3 ROBMaestroChannel|—  
4 ROBTicWaistController|—  
5 ROBAmberArmGatewayClient|—  
6 ROBControllerCommandArbiter
```

Move parsing and mapping into pure functions with fixtures first. Preserve the Objective-C façade so Interface Builder and existing callers keep working. Introduce typed state snapshots. Move every descriptor to one owning queue. Rate-limit diagnostics. Remove Head/Torso UI only after confirming no nib binding or operator workflow still requires it.

For camera work, preserve `CameraManagerProtocol` and `CameraFrameSet`. For model work, preserve `ROBLocalImprovisationProviding`, `SceneSnapshot`, and strict codecs. For Vision Pro work, preserve the versioned control/video contracts and update both repositories in lockstep.

File-by-file reading route

Read in this order and make notes beside the code:

1. `README.md` and `docs/` for declared behavior and operational boundaries.
2. `AppDelegate.m` and `ROBMainViewController.mm` for process composition.
3. `ROBSerialBox.h/.m` for hardware history, current Base discovery, Maestro/Tic, and arm seams.
4. `CameraManager.swift` and `Webcam_color.py` for provider ownership and RGB-D framing.
5. `CameraViewController.swift`, pose detector, and skeleton renderer for perception/display cost.
6. `ROBSceneSnapshot.swift` for typed world state and confidence.
7. `ROBMLXRuntime.swift`, MLX provider, and local-improvisation protocol for private inference.
8. `GeminiRoboticsProtocol.swift` before `ROBAI.swift`; learn the wire types before the session actor.
9. `ROBStageShowProtocol.swift` before its coordinator; learn allowed data before lifecycle.
10. `ROBSaberChoreography.swift`, AMBER kinematics, and visual calibration together.
11. `ROBVideoProtocol.swift`, encoder, and server, followed by Volume 7's receiver chapters.
12. Every matching fixture test before making a change.

Production review checklist

- ☐ Every serial descriptor has one owner and bounded reconnect behavior.
- ☐ Base identity is detected by its existing firmware response without probe commands.
- ☐ Stale controller input produces one neutral frame and then drops heartbeat.
- ☐ Commanded, measured, and visually estimated poses remain distinct.
- ☐ Camera ownership cannot race between DepthAI and UVC providers.
- ☐ Every IPC and network length is validated before allocation.
- ☐ Slow perception, inference, encoding, and network consumers retain newest-only bounded work.
- ☐ Vision and model work never blocks controller or main-thread responsiveness.
- ☐ SceneKit nodes, contexts, pixel buffers, and model containers remain bounded over long runs.
- ☐ Local and cloud model output crosses strict typed codecs.
- ☐ No language model has direct motor, shell, serial, or arm authority.
- ☐ Stage cues have deadlines, cancellation correlation, and authored fallback.
- ☐ Camera and microphone streaming show explicit effective operator state.
- ☐ Vision Pro control and video protocol versions match Volume 7's client.
- ☐ Historical Kinect artifacts are documented before removal or reuse.
- ☐ Tests pass before any powered-hardware commissioning.

Closing principle

Cerebro is strongest when every subsystem admits uncertainty and owns failure locally. Serial discovery should identify rather than guess. Depth should mark invalid pixels. Vision should preserve confidence. Models should propose typed meaning rather than emit actuator bytes. Stage shows should continue through bounded fallback. Video should drop frames rather than delay control. The Vision Pro should express fresh human intent, while Cerebro and the robot enforce authority, time, and safe limits.

That is how a mixed Swift and Objective-C codebase becomes a dependable robot mind: not through one magical model, but through explicit contracts between imperfect parts.